App Development Project Report

PLACEMENT PORTAL APPLICATION (PPA) — VERSION 2 REPORT

1. Student Details

Name: Dhruv Jyoti Das
Roll Number: 23f2001463
Email: 23f2001463@ds.study.iitm.ac.in
About Me: I am a 4th-year Undergraduate student in Computer Science and Engineering at SRMIST, Kattankulathur, and a Diploma student in the BS Data Science program at IIT Madras. Beyond my coursework, I am a research enthusiast currently collaborating on multiple research projects with faculty from IIT Indore and universities in Ireland and Japan, while also pursuing my own independent research. My interests lie in web application development, data-driven technologies, and advancing into a dedicated research career.

---

2. Project Details

Project Title: Placement Portal Application (PPA) — Version 2

Problem Statement:

Institutes rely on spreadsheets, emails, and manual coordination to manage campus recruitment, making it difficult to track company approvals, placement drives, student registrations, and application status. This project designs and builds a role-based web application where the Admin (Institute), Companies, and Students each interact with a single system to manage the full placement lifecycle — from company registration and drive approval through student application, shortlisting, and final placement.

Approach:

The system architecture utilizes **Flask** to power a backend REST API, logically partitioned into role-specific Blueprints for administrators, corporate entities, and students. The user interface is driven by **Vue 3** (utilizing Single File Components and Vite), which interacts with the server through JWT-secured endpoints. All data persistence is handled by **SQLite** via programmatically defined SQLAlchemy models. Furthermore, **Redis** serves as the backbone for both the caching layer (optimizing dashboard metrics and drive data) and the **Celery** task queue, which manages automated reminders, periodic reporting, and asynchronous CSV generation, while SMTP integration with MailHog facilitates email simulation for demonstration purposes.

3. AI/LLM Declaration

I utilized a suite of AI tools, including Claude and Codex, throughout the development process. These tools assisted with designing the database schema, writing documentation comments, UI design, and providing strategies and tips for debugging. While the core application logic and implementation were authored by me, AI tools were used to modify and refine approximately 35-40% of the codebase, ensuring robust implementation and optimization.

---

4. Technologies and Frameworks Used

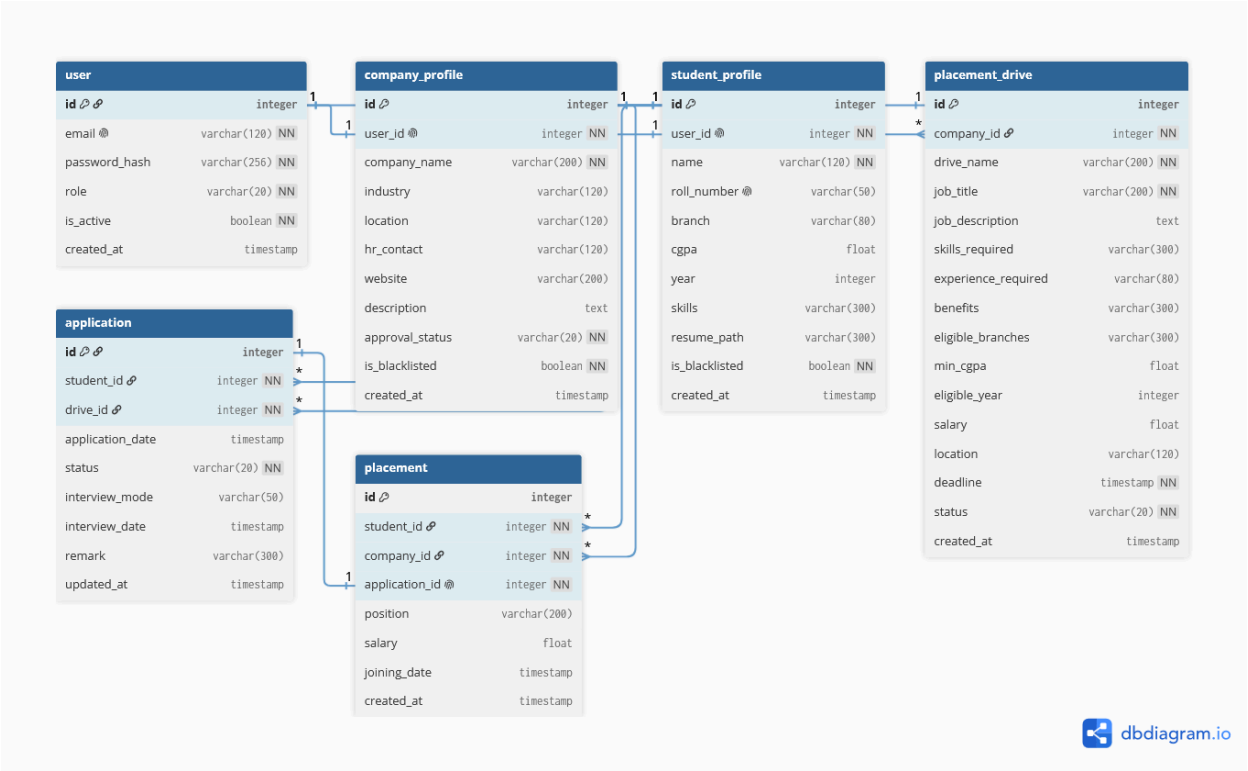
Technology/Framework	Description
Flask	Core backend REST API framework
Flask-SQLAlchemy	ORM for the SQLite database
Flask-JWT-Extended	Token-based authentication and role-based access control
Flask-Caching	Redis-backed caching for dashboard stats and drive listings
Flask-CORS	Allows the Vue dev server to call the Flask API cross-origin
Celery	Background job queue — scheduled reminders, monthly reports, async CSV export
Redis	Broker/backend for Celery, and the cache store for Flask-Caching
SQLite	Lightweight database, created programmatically via SQLAlchemy
Vue 3	Frontend framework (Composition API, Single File Components)
Vite	Frontend build tool and dev server
Vue Router	Client-side routing with role-based navigation guards
Bootstrap 5	Frontend styling — the only CSS framework used
Python smtplib + MailHog	SMTP email delivery to a local mail catcher for demo purposes

5. Database Schema / ER Diagram

Table Name	Fields
User	id, email, password_hash, role (admin/company/student), is_active, created_at
CompanyProfile	id, user_id, company_name, industry, location, hr_contact, website, description, approval_status, is_blacklisted
StudentProfile	id, user_id, name, roll_number, branch, cgpa, year, skills, resume_path, is_blacklisted
PlacementDrive	id, company_id, drive_name, job_title, job_description, skills_required, experience_required, benefits, eligible_branches, min_cgpa, eligible_year, salary, location, deadline, status
Application	id, student_id, drive_id, application_date, status, interview_mode, interview_date, remark
Placement	id, student_id, company_id, application_id, position, salary, joining_date

Database Relationships:

Relationship Type	Description
One-to-One	User ↔ CompanyProfile, User ↔ StudentProfile
One-to-Many	CompanyProfile → PlacementDrive
One-to-Many	StudentProfile → Application, PlacementDrive → Application
One-to-One	Application → Placement (Created automatically upon selection)



---

6. API Resource Endpoints

Endpoint	Method	Description
/api/register	POST	Registration for students or companies (excludes administrative self-registration)
/api/login	POST	Authentication endpoint returning JWT and user role
/api/admin/stats	GET	Retrieves cached dashboard statistics for administration
/api/admin/companies	GET	Search or list registered companies, filterable by industry
/api/admin/companies/:id/:action	PUT	Administer company status: approve, reject, or manage blacklisting

Endpoint	Method	Description
/api/admin/companies/:id	DELETE	Permanent removal of a company profile
/api/admin/students	GET	Search or list student profiles, including contact details
/api/admin/drives	GET	Comprehensive search and listing of all placement drives
/api/admin/drives/:id/:action	PUT	Update drive status: approval, rejection, or closure
/api/admin/drives/:id	DELETE	Permanent removal of a placement drive record
/api/company/drives	POST	Initialize a placement drive (requires prior admin approval)
/api/company/applications/:id	PUT	Manage applicant progress, schedule interviews, and finalize selections
/api/student/drives	GET	Browse eligibility-filtered approved drives (utilizes Redis caching)
/api/student/drives/:id/apply	POST	Apply for a drive with eligibility and duplicate validation
/api/student/export	POST	Initialize asynchronous CSV export of individual placement history

---

7. Video Presentation

Drive Link:

Placement_Portal_23f2001463_DhruvJyotiDas.mp4
https://drive.google.com/file/d/1Z3yrk8uz_l4FfzxBkS-agoA0pGwp_mEo/view?usp=sharing

---